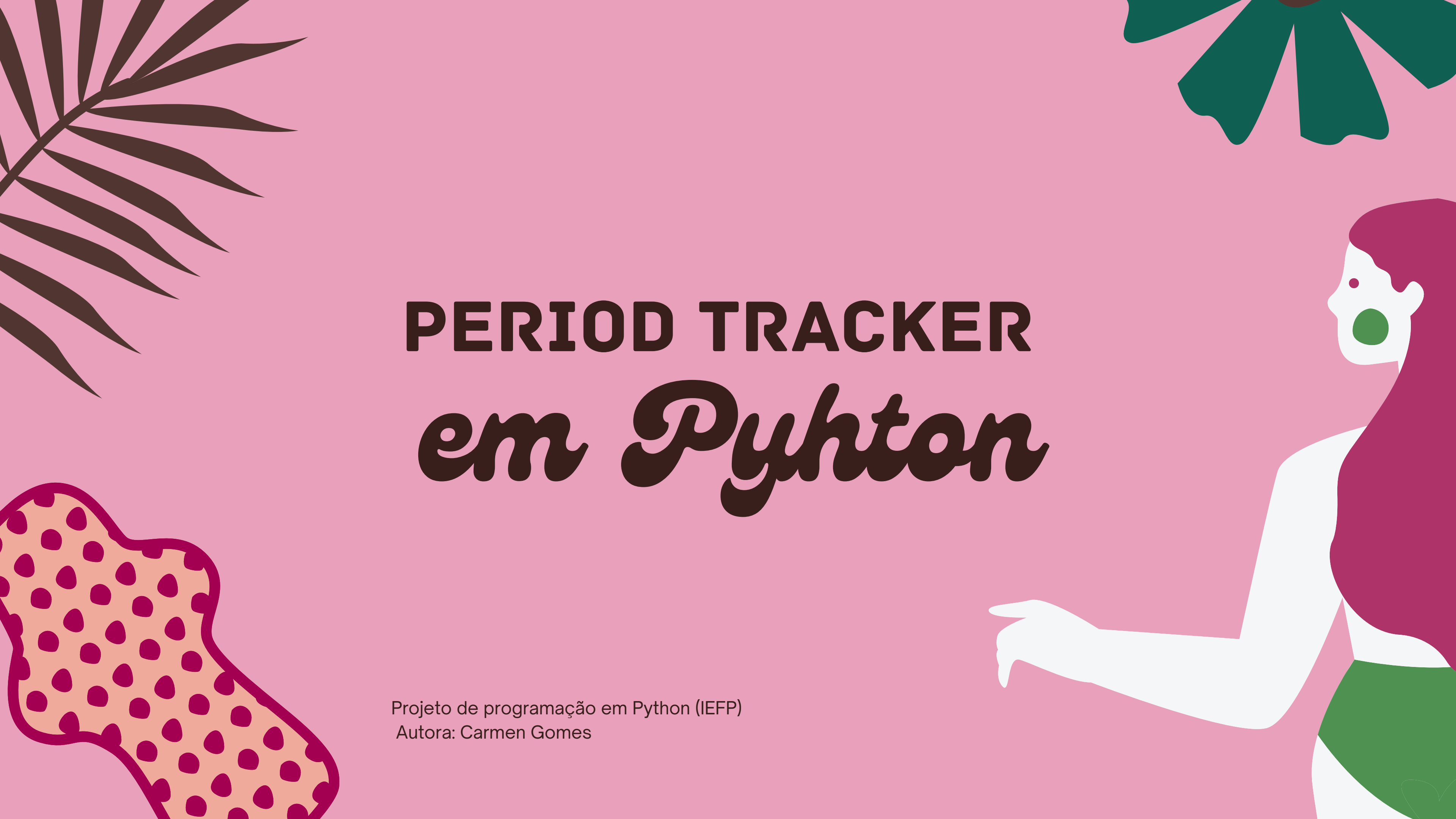




PERIOD TRACKER *em Python*

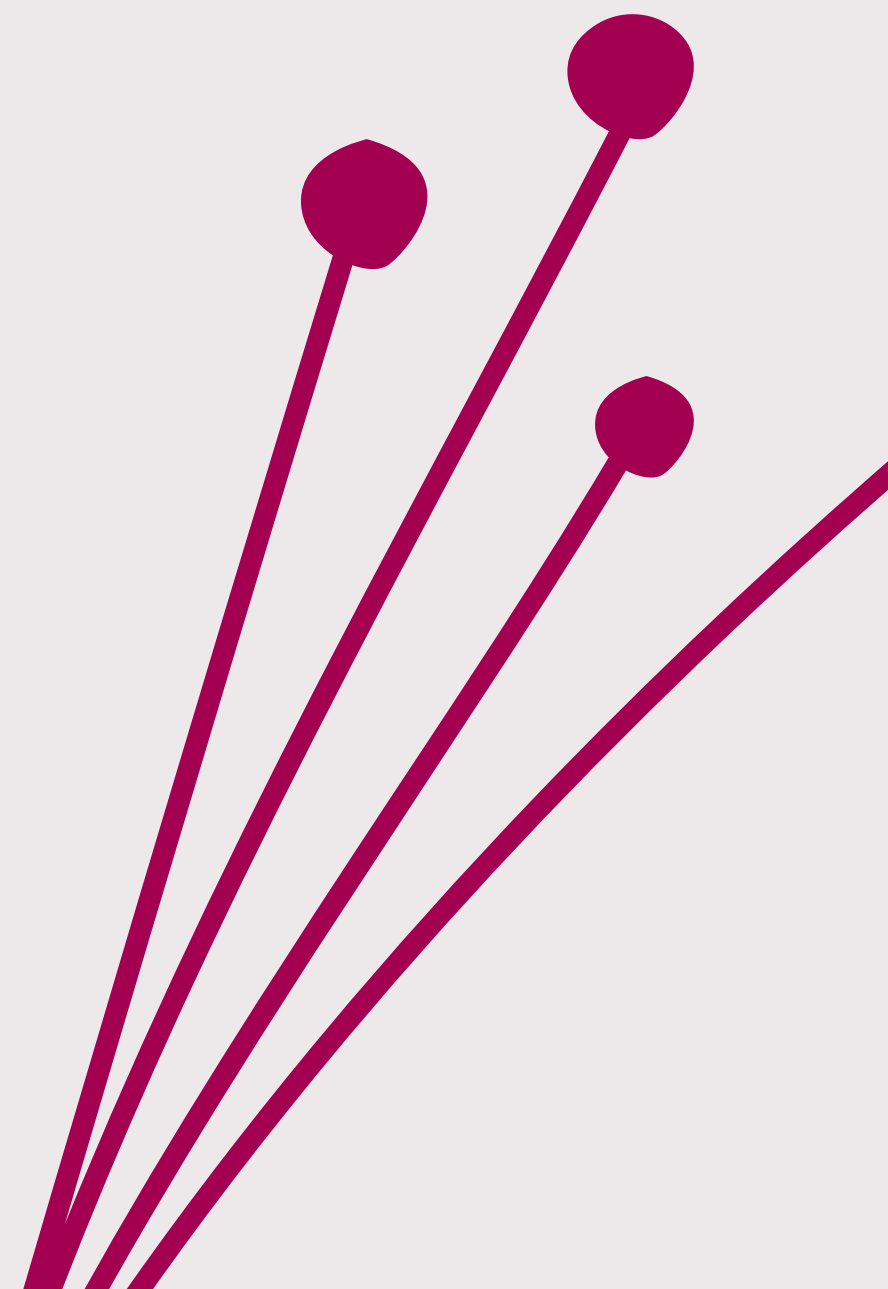
Projeto de programação em Python (IEFP)
Autora: Carmen Gomes

A Ideia e a Otimização de UX

A sua origem parte da simplificação de uma ideia pessoal para o curso do IEFP.

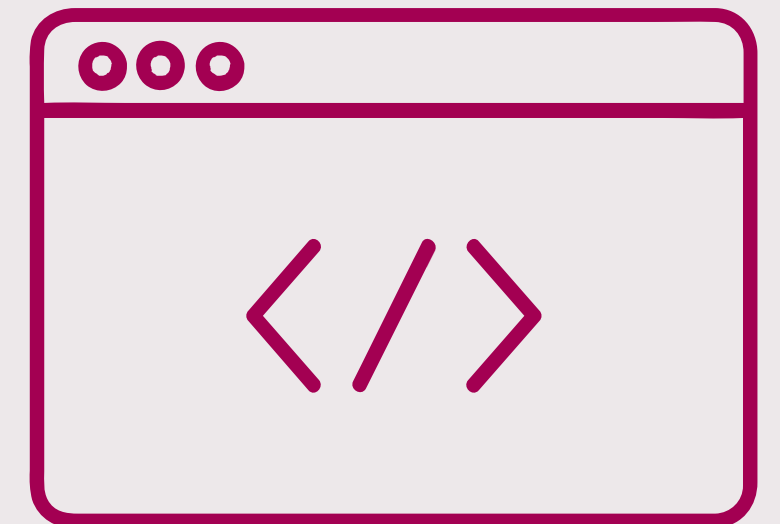
O **objetivo**? Ajudar as utilizadoras a perceber o corpo sem a complexidade das apps comerciais.

Um dos pormenores também é o foco na **Usabilidade**, ter menus numéricos simples no terminal e perguntas inteligentes que não se repetem se a fase do ciclo já mudou.

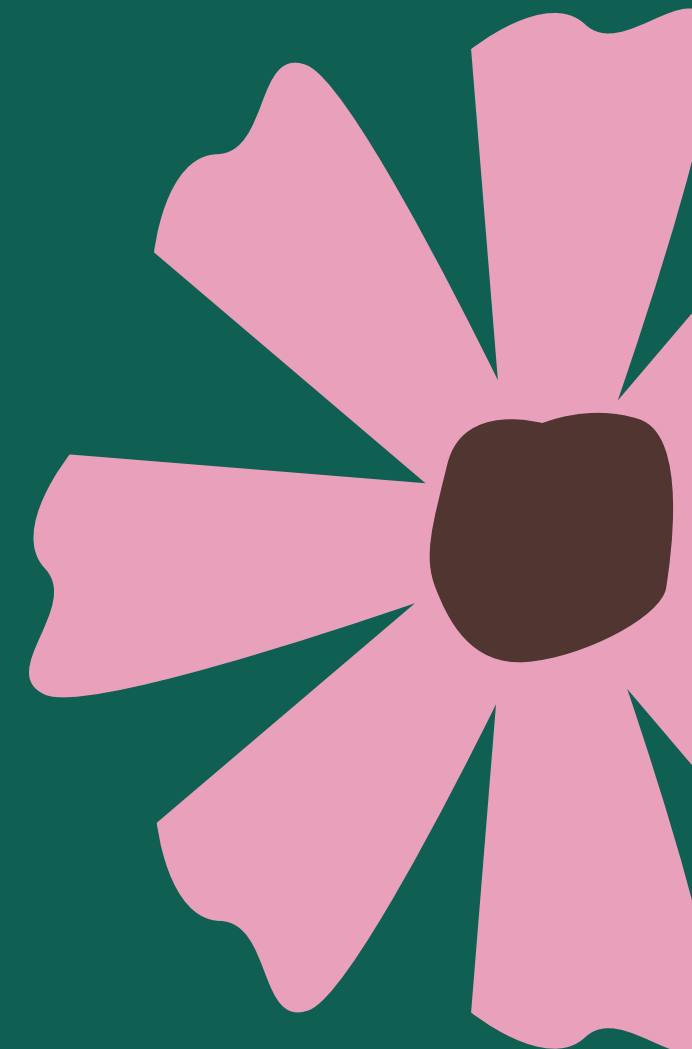
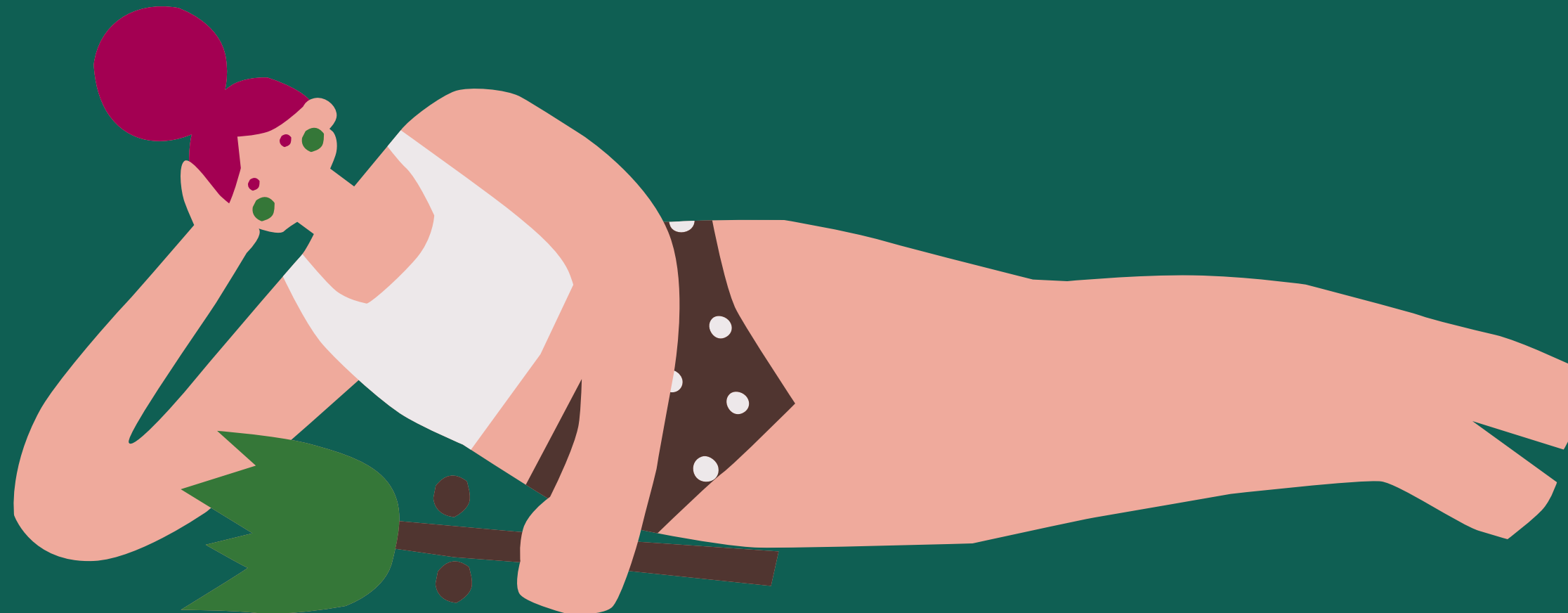
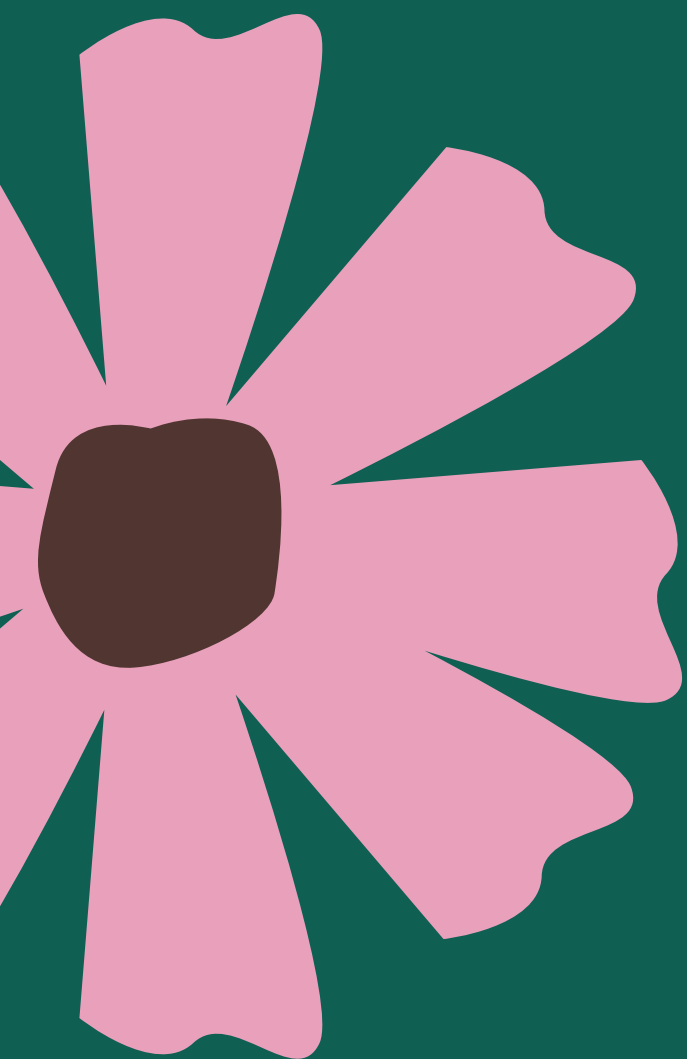


Arquitetura e Organização do Código

- Estrutura Modular (Separação de Responsabilidades):
 - **main.py**: Operador central e loop contínuo do menu.
 - **ler_gravar.py**: Persistência (leitura e escrita isolada no JSON).
 - **ciclo.py**: Regras das fases e resumos explicativos.
 - **saude.py**: Motor de regras de saúde e triagem por cores ANSI.
- Ficheiros de Dados: dados_ciclo.json e medicos.json.



COMO *Funciona*



O Fluxo Inicial (O Primeiro Arranque)

- **O que acontece no ecrã:** O programa dá as boas-vindas e deteta que o ficheiro dados_ciclo.json está vazio.
- **Ação do Utilizador:** Introduz Nome, Idade e Data da última menstruação.
- **Bastidores Técnicos (ler_gravar.py):** * A função inicializar_perfil valida os dados e guarda-os imediatamente no JSON.
 - Usa o símbolo = para atribuição correta de variáveis.



O Loop Diário e a Inteligência de UX

(O CORAÇÃO DO MAIN)

- **O que acontece no ecrã:** Um menu contínuo (*Loop while*) permite registar vários dias seguidos de uma só vez, sem fechar a aplicação.
- **Demonstração do Código:**
 - No Dia 1, o programa pergunta obrigatoriamente se a utilizadora está menstruada.
 - Se no Dia 5 ela responder "Não", o programa guarda a transição.
 - No Dia 6, o programa usa o índice [-1] para espreitar o dia anterior no JSON. Como a fase já mudou, ele salta a pergunta automaticamente e vai direto para os sintomas gerais.
- **Pedagogia:** Antes das perguntas, o módulo ciclo.py exibe uma frase explicativa em azul (\033[94m) sobre o estado do corpo naquela fase.

O Motor de Análise de Saúde e Alertas Visuais

- **O que acontece no ecrã:** O utilizador escolhe a "Opção 2: Ver Relatório de Saúde".
- Parte Técnica (saude.py):
 - O programa faz uma “varredura” estatística na lista de registos do JSON.
 - Avalia os parâmetros através de uma árvore de decisão (if/elif).
- **Formatos de Alerta (Cores ANSI):**
 - **Verde:** Ciclo dentro dos parâmetros normais.
 - **Amarelo:** Alerta moderado (ex: fluxo pesado persistente).
 - **Vermelho:** Alerta máximo (ex: menstruação prolongada > 7 dias ou múltiplos dias com vómitos).

Triagem e Recomendação Geográfica

(MEDICOS.JSON)

- **O que acontece no ecrã:** O sistema deteta anomalias e sugere especialidades. Pergunta pela localização (Lisboa/Porto) e exhibe os contactos médicos.
- **Resolução do Bug Técnico (Multi-Especialidades):**
 - Diagnóstico do Erro: "TypeError" ao tentar usar uma lista mutável como chave de dicionário.
 - Solução: Integração de um loop 'for' para iterar o conjunto (set) de especialidades e extrair os médicos um a um.
 - Localização anexada permanentemente ao perfil no JSON para evitar perguntas redundantes.

Conclusão

Temos uma aplicação modular estável, persistente e focada na experiência do utilizador, cuja arquitetura permite uma divisão clara entre src/ (produção) e tests/ (validação).



Obrigada!